

Kathmandu University

Department of Computer Science and Engineering
Dhulikhel, Kavre



"Self-Healing Firmware Architecture for Fault-Tolerant Embedded Systems using In-Application Programming"

Date: August 4, 2026

Presentation Outline

- 1 The Team
- 2 Introduction
- 3 Design and Implementation
- 4 Implementation Details
- 5 Achievements
- 6 Conclusion and Recommendations
- 7 References



Meet The Team

The Team



Sushil Nepal

Project Supervisor

The Team



Sushil Nepal

Project Supervisor



Rajat Dahal

The Team



Sushil Nepal
Project Supervisor



Rajat Dahal



Suvesh Gurung

The Team



Sushil Nepal

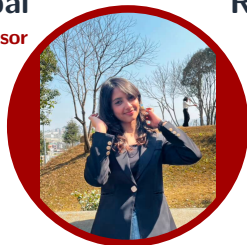
Project Supervisor



Rajat Dahal



Suvesh Gurung



Pramisha Sapkota

The Team



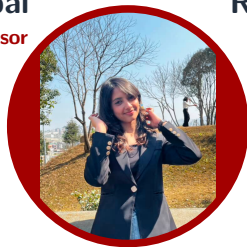
Sushil Nepal
Project Supervisor



Rajat Dahal



Suvesh Gurung



Pramisha Sapkota



Nischal Subedi



Introduction

Background & Motivation

- **Growing Dependence on Embedded Systems:** Low-cost microcontrollers such as the STM32F103C8T6 are widely deployed in industrial control, remote sensor networks, and space applications like CubeSats.

Background & Motivation

- **Growing Dependence on Embedded Systems:** Low-cost microcontrollers such as the STM32F103C8T6 are widely deployed in industrial control, remote sensor networks, and space applications like CubeSats.
- **Firmware Reliability Challenge:** Flash memory is vulnerable to **soft errors / bit flips** caused by cosmic radiation or electromagnetic interference, corrupting the running application.

Background & Motivation

- **Growing Dependence on Embedded Systems:** Low-cost microcontrollers such as the STM32F103C8T6 are widely deployed in industrial control, remote sensor networks, and space applications like CubeSats.
- **Firmware Reliability Challenge:** Flash memory is vulnerable to **soft errors / bit flips** caused by cosmic radiation or electromagnetic interference, corrupting the running application.
- **Need for Autonomous Recovery:** Devices in remote or unreachable locations (e.g. LEO satellites) cannot rely on OTA patches or physical reflashing, making onboard self-recovery essential.

Background & Motivation

- **Growing Dependence on Embedded Systems:** Low-cost microcontrollers such as the STM32F103C8T6 are widely deployed in industrial control, remote sensor networks, and space applications like CubeSats.
- **Firmware Reliability Challenge:** Flash memory is vulnerable to **soft errors / bit flips** caused by cosmic radiation or electromagnetic interference, corrupting the running application.
- **Need for Autonomous Recovery:** Devices in remote or unreachable locations (e.g. LEO satellites) cannot rely on OTA patches or physical reflashing, making onboard self-recovery essential.

Project Objective

Develop a self-healing firmware architecture for the STM32F103C8T6 microcontroller that detects firmware corruption using a CRC-based integrity check and autonomously restores a valid application through In-Application Programming (IAP).

Problem Scenario

- Firmware executes normally after deployment.

Problem Scenario

- Firmware executes normally after deployment.
- A bit flip or environmental fault corrupts the application code in flash.

Problem Scenario

- Firmware executes normally after deployment.
- A bit flip or environmental fault corrupts the application code in flash.
- Conventional reset / watchdog mechanisms cannot repair corrupted firmware – they simply re-run the broken code.

Problem Scenario

- Firmware executes normally after deployment.
- A bit flip or environmental fault corrupts the application code in flash.
- Conventional reset / watchdog mechanisms cannot repair corrupted firmware – they simply re-run the broken code.
- A custom bootloader verifies integrity at every boot and restores the application from an internal backup image.

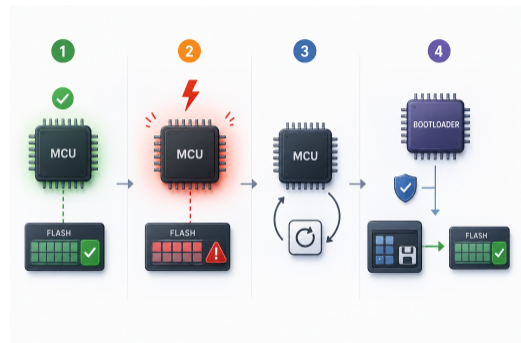


Figure: Rough representation of what happens

Design and Implementation

Flash Memory Architecture

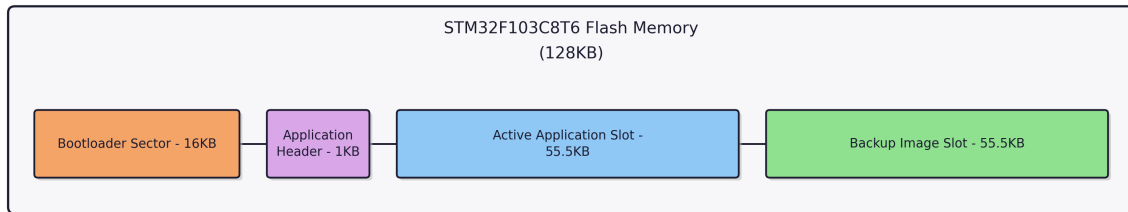


Figure: Quadripartition memory structure

CRC Verification Flowchart

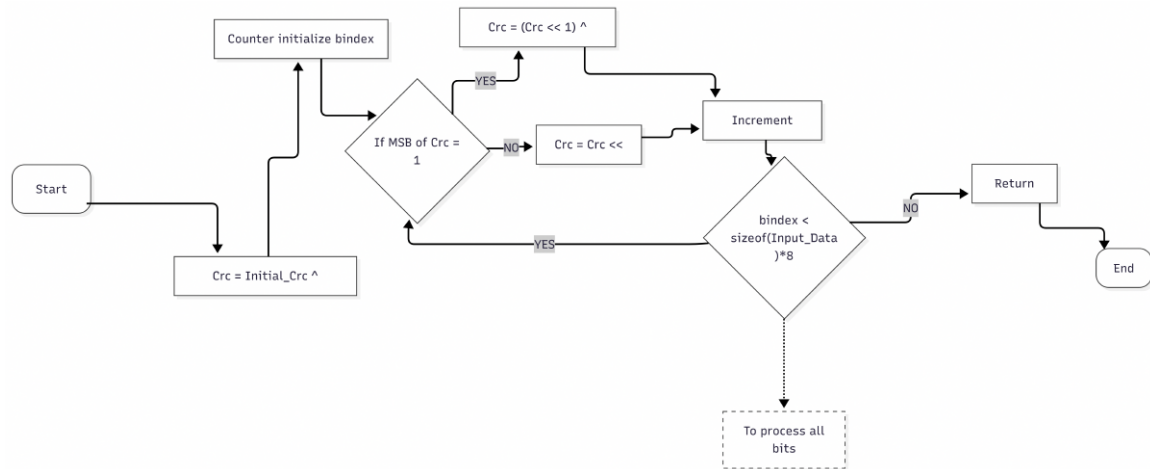


Figure: Flowchart of CRC calculation unit. Source: [?]

System Architecture (1/2)

System Architecture (1/2)

Architectural Pattern: Secure-boot architecture with the bootloader acting as the root of trust.

System Architecture (1/2)

Architectural Pattern: Secure-boot architecture with the bootloader acting as the root of trust.

Bootloader Layer

- Entry point at 0x0800 0000, 16KB allocation.
- Initializes the hardware CRC peripheral.
- Verifies application integrity before every execution cycle.

System Architecture (1/2)

Architectural Pattern: Secure-boot architecture with the bootloader acting as the root of trust.

Bootloader Layer

- Entry point at 0x0800 0000, 16KB allocation.
- Initializes the hardware CRC peripheral.
- Verifies application integrity before every execution cycle.

Memory Layout

- Application Header (1KB): stores CRC & firmware size.
- Active Application Slot (55.5KB): verified, executing firmware.
- Backup Image Slot (55.5KB): read-only “golden image”.

System Architecture (2/2)

System Architecture (2/2)

Core Modules & Interfaces

System Architecture (2/2)

Core Modules & Interfaces

Verification & Decision

- Verification Module: hardware CRC unit generates a signature of active memory.
- Decision Engine: compares the signature against the stored “golden hash”.

System Architecture (2/2)

Core Modules & Interfaces

Verification & Decision

- Verification Module: hardware CRC unit generates a signature of active memory.
- Decision Engine: compares the signature against the stored “golden hash”.

Recovery & Reporting

- IAP Recovery Module: drives the FPEC to erase & reprogram the active slot.
- Telemetry Module: reports health & recovery logs over UART.



Pipelines: Verification & Recovery Pipeline

The Verification Pipeline

The Verification Pipeline

- **Header Fetch:** Bootloader reads the firmware size and reference **CRC** from the **Application Header**.

The Verification Pipeline

- **Header Fetch:** Bootloader reads the firmware size and reference **CRC** from the **Application Header**.
- **Signature Calculation:** The hardware **CRC peripheral** computes a signature over the Active Application Slot, bounded by the stored size.

The Verification Pipeline

- **Header Fetch:** Bootloader reads the firmware size and reference **CRC** from the **Application Header**.
- **Signature Calculation:** The hardware **CRC peripheral** computes a signature over the Active Application Slot, bounded by the stored size.
- **Decision:** The calculated value is compared against the stored reference to choose “boot normally” or “trigger recovery”.

The Verification Pipeline

- **Header Fetch:** Bootloader reads the firmware size and reference **CRC** from the **Application Header**.
- **Signature Calculation:** The hardware **CRC peripheral** computes a signature over the Active Application Slot, bounded by the stored size.
- **Decision:** The calculated value is compared against the stored reference to choose “boot normally” or “trigger recovery”.

Fast-Path Boot

When the CRC values match, recovery is skipped entirely – the bootloader sets the **Main Stack Pointer** and jumps straight to the application, keeping boot overhead minimal.

The Recovery Pipeline

The Recovery Pipeline

- **Page Erasure:** Number of 1KB pages is computed from the stored application size and erased via the **FPEC**, then verified.

The Recovery Pipeline

- **Page Erasure:** Number of 1KB pages is computed from the stored application size and erased via the **FPEC**, then verified.
- **Half-Word Programming:** Each 32-bit word from the backup image is split into two 16-bit half-words and written into the Active Slot.

The Recovery Pipeline

- **Page Erasure:** Number of 1KB pages is computed from the stored application size and erased via the **FPEC**, then verified.
- **Half-Word Programming:** Each 32-bit word from the backup image is split into two 16-bit half-words and written into the Active Slot.
- **Re-verification:** A second CRC check confirms the restored image before application handover.

The Recovery Pipeline

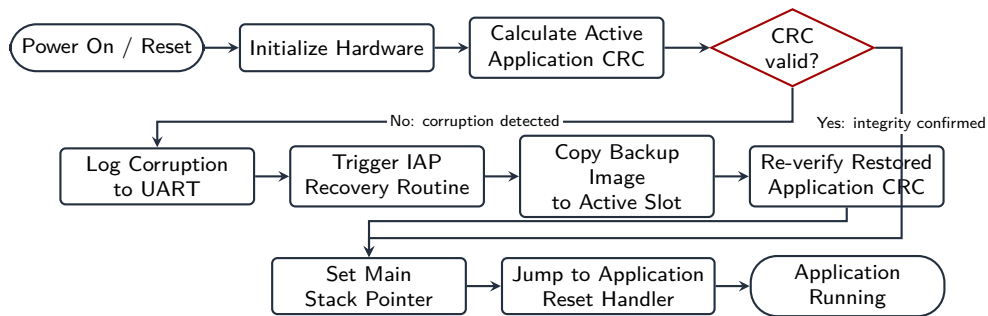
- **Page Erasure:** Number of 1KB pages is computed from the stored application size and erased via the **FPEC**, then verified.
- **Half-Word Programming:** Each 32-bit word from the backup image is split into two 16-bit half-words and written into the Active Slot.
- **Re-verification:** A second CRC check confirms the restored image before application handover.

Fail-Safe by Design

Recovery writes are confined to the **Active Application Slot** only – the read-only **Backup Image** is never touched, keeping a reliable recovery source available for future cycles.

System Architecture Flow

Everything working together.





Implementation Details

Integrity Check: Why Hardware CRC?

- **Problem with Software Checksums:** Pure software checksum loops consume CPU cycles and code space on a resource-constrained MCU.

Integrity Check: Why Hardware CRC?

- **Problem with Software Checksums:** Pure software checksum loops consume CPU cycles and code space on a resource-constrained MCU.
- **Chosen Solution:** STM32's on-chip **Hardware CRC Peripheral**.

Integrity Check: Why Hardware CRC?

- **Problem with Software Checksums:** Pure software checksum loops consume CPU cycles and code space on a resource-constrained MCU.
- **Chosen Solution:** STM32's on-chip **Hardware CRC Peripheral**.
- **Benefits:**
 - Fast, deterministic signature generation.
 - Size-bounded verification via the stored `app_size`.
 - Native HAL integration (`HAL_CRC_Calculate`).

Flash Programming & Fault Simulation

- **Programming Interface:** Writes to the **FPEC** in half-word (16-bit) units, as required by the STM32F103 flash architecture.

Flash Programming & Fault Simulation

- **Programming Interface:** Writes to the **FPEC** in half-word (16-bit) units, as required by the STM32F103 flash architecture.
- **Fault Injection:** Corruption is simulated by manually editing bytes in the Active Application Slot using **STM32CubeProgrammer**.

Flash Programming & Fault Simulation

- **Programming Interface:** Writes to the **FPEC** in half-word (16-bit) units, as required by the STM32F103 flash architecture.
- **Fault Injection:** Corruption is simulated by manually editing bytes in the Active Application Slot using **STM32CubeProgrammer**.
- **Scope Isolation:** Recovery writes are restricted to the active slot, keeping the bootloader, header, and backup image untouched during testing.

Project Achievements

Project Achievements

Project Achievements

- ✓ **Flash Memory Partitioning:** Implemented a linker-defined layout separating the bootloader, application header, active application, and backup image.

Project Achievements

- ✓ **Flash Memory Partitioning:** Implemented a linker-defined layout separating the bootloader, application header, active application, and backup image.
- ✓ **Corruption Detection:** CRC-based bootloader verification reliably distinguishes an intact image from an injected fault.

Project Achievements

- ✓ **Flash Memory Partitioning:** Implemented a linker-defined layout separating the bootloader, application header, active application, and backup image.
- ✓ **Corruption Detection:** CRC-based bootloader verification reliably distinguishes an intact image from an injected fault.
- ✓ **Autonomous Recovery via IAP:** Flash driver erases and reprograms the active slot from the internal backup image without host intervention.

Project Achievements

- ✓ **Flash Memory Partitioning:** Implemented a linker-defined layout separating the bootloader, application header, active application, and backup image.
- ✓ **Corruption Detection:** CRC-based bootloader verification reliably distinguishes an intact image from an injected fault.
- ✓ **Autonomous Recovery via IAP:** Flash driver erases and reprograms the active slot from the internal backup image without host intervention.
- ✓ **Full Self-Healing Validation:** End-to-end detect → recover → revalidate → execute sequence demonstrated under simulated corruption.

Project Achievements

- ✓ **Flash Memory Partitioning:** Implemented a linker-defined layout separating the bootloader, application header, active application, and backup image.
- ✓ **Corruption Detection:** CRC-based bootloader verification reliably distinguishes an intact image from an injected fault.
- ✓ **Autonomous Recovery via IAP:** Flash driver erases and reprograms the active slot from the internal backup image without host intervention.
- ✓ **Full Self-Healing Validation:** End-to-end detect → recover → revalidate → execute sequence demonstrated under simulated corruption.
- ✓ **Normal-Boot Efficiency:** Verified that an unmodified application boots directly without unnecessary flash rewrites.

Conclusion and Recommendations

Conclusion

Conclusion

- **Beyond Reset-Based Recovery:** Successfully implemented a bootloader-level self-healing mechanism that repairs corrupted code, rather than merely restarting it like a watchdog timer.

Conclusion

- **Beyond Reset-Based Recovery:** Successfully implemented a bootloader-level self-healing mechanism that repairs corrupted code, rather than merely restarting it like a watchdog timer.
- **Functional Validation:** Confirmed through controlled fault injection that corrupted firmware is consistently detected and autonomously restored.

Conclusion

- **Beyond Reset-Based Recovery:** Successfully implemented a bootloader-level self-healing mechanism that repairs corrupted code, rather than merely restarting it like a watchdog timer.
- **Functional Validation:** Confirmed through controlled fault injection that corrupted firmware is consistently detected and autonomously restored.
- **Reliable Foundation:** Established a four-region flash layout (bootloader, header, active app, backup) suited to remote and space-based deployments.

Conclusion

- **Beyond Reset-Based Recovery:** Successfully implemented a bootloader-level self-healing mechanism that repairs corrupted code, rather than merely restarting it like a watchdog timer.
- **Functional Validation:** Confirmed through controlled fault injection that corrupted firmware is consistently detected and autonomously restored.
- **Reliable Foundation:** Established a four-region flash layout (bootloader, header, active app, backup) suited to remote and space-based deployments.

Current Technical Constraints:

Conclusion

- **Beyond Reset-Based Recovery:** Successfully implemented a bootloader-level self-healing mechanism that repairs corrupted code, rather than merely restarting it like a watchdog timer.
- **Functional Validation:** Confirmed through controlled fault injection that corrupted firmware is consistently detected and autonomously restored.
- **Reliable Foundation:** Established a four-region flash layout (bootloader, header, active app, backup) suited to remote and space-based deployments.

Current Technical Constraints:

- **Unprotected Recovery Path:** The bootloader, header, and backup image themselves cannot be repaired if corrupted.

Conclusion

- **Beyond Reset-Based Recovery:** Successfully implemented a bootloader-level self-healing mechanism that repairs corrupted code, rather than merely restarting it like a watchdog timer.
- **Functional Validation:** Confirmed through controlled fault injection that corrupted firmware is consistently detected and autonomously restored.
- **Reliable Foundation:** Established a four-region flash layout (bootloader, header, active app, backup) suited to remote and space-based deployments.

Current Technical Constraints:

- **Unprotected Recovery Path:** The bootloader, header, and backup image themselves cannot be repaired if corrupted.
- **Flash Capacity & Endurance:** Storing a full backup image halves usable application space and consumes limited erase/program cycles.

Conclusion

- **Beyond Reset-Based Recovery:** Successfully implemented a bootloader-level self-healing mechanism that repairs corrupted code, rather than merely restarting it like a watchdog timer.
- **Functional Validation:** Confirmed through controlled fault injection that corrupted firmware is consistently detected and autonomously restored.
- **Reliable Foundation:** Established a four-region flash layout (bootloader, header, active app, backup) suited to remote and space-based deployments.

Current Technical Constraints:

- **Unprotected Recovery Path:** The bootloader, header, and backup image themselves cannot be repaired if corrupted.
- **Flash Capacity & Endurance:** Storing a full backup image halves usable application space and consumes limited erase/program cycles.
- **Integrity, Not Authenticity:** CRC detects accidental corruption but does not protect against a deliberate, coordinated modification.

Future Recommendations

- **Resilience & Security**

- **Protected Metadata:** Hardware write-protection for the bootloader and redundant, versioned application headers.
- **Secure Boot:** Replace CRC with cryptographic hashes / digital signatures for authenticated firmware execution.

Future Recommendations

- **Resilience & Security**

- **Protected Metadata:** Hardware write-protection for the bootloader and redundant, versioned application headers.
- **Secure Boot:** Replace CRC with cryptographic hashes / digital signatures for authenticated firmware execution.

- **Fault Tolerance**

- **Multiple Backups:** Store several verified backup copies to survive corruption of a single backup image.
- **Transactional Recovery:** Record recovery progress so interrupted flashing can resume safely after power loss.

Future Recommendations

- **Resilience & Security**

- **Protected Metadata:** Hardware write-protection for the bootloader and redundant, versioned application headers.
- **Secure Boot:** Replace CRC with cryptographic hashes / digital signatures for authenticated firmware execution.

- **Fault Tolerance**

- **Multiple Backups:** Store several verified backup copies to survive corruption of a single backup image.
- **Transactional Recovery:** Record recovery progress so interrupted flashing can resume safely after power loss.

- **Deployment & Monitoring**

- **Telemetry Logging:** Persist recovery events over UART for post-deployment analysis.
- **Authenticated OTA:** Securely refresh the backup image whenever a communication link is available.



References

References

-  Zhou, W. (2015). *Compact Firmware Upgrader for Robot System Based on IAP Methodology*. PhD thesis, Cornell University.
-  Neves, B. P., Santos, V. D. N., and Valente, A. (2024). *Innovative Firmware Update Method to Microcontrollers During Runtime*. *Electronics*, 13(7).
-  STMicroelectronics (2012). *PM0075: STM32F10xxx Flash Memory Programming Manual*, revision 2. Document ID 17863 Rev 2.
-  STMicroelectronics (2022). *AN4187: Using the CRC Peripheral on STM32 Microcontrollers*, rev. 2. Application Note.

Thank You!

Questions?

Project Repository:

<https://github.com/suveshgurung/Self-Healing-Firmware-Architecture-for-STM32F103C8T6>